00_SheetDb.gs

Source path: ‘sheet_db_lib/00_SheetDb.gs‘

‘‘‘javascript
```
/**
 * @fileoverview Public, library-visible exports for SheetDb.
 */

/**
 * Library semantic version.
 *
 * NOTE: Keep this in sync with release tags when publishing as an Apps Script library.
 * @type {string}
 */
var SHEET_DB_VERSION = '0.1.0';

/**
 * Creates a new SheetDb client instance.
 *
 * @param {Object=} options Client configuration options.
 * @return {DbClient}
 */
function _sheetDbCreateClient(options) {
  return new DbClient(new Config(options || {}));
}

/**
 * Bootstraps a client with one or more table schemas.
 *
 * @param {Object=} options
 * @param {Array<{name: string, schema: Object}>=} options.tables Tables to register.
 * @param {Object=} options.clientOptions Configuration passed to ‘createClient‘.
 * @return {DbClient}
 */
function _sheetDbBootstrap(options) {
  var opts = options || {};
  var client = _sheetDbCreateClient(opts.clientOptions || {});
  var tables = opts.tables || [];

  tables.forEach(function(table) {
    if (table && table.name && table.schema) {
      client.registerTable(table.name, table.schema);
    }
  });

  return client;
}

/**
 * Basic health check that can be called by consuming projects.
 *
 * @return {{ok: boolean, version: string, timestamp: string, details: Object}}
 */
function _sheetDbHealthCheck() {
  try {
    var client = _sheetDbCreateClient();
    return {
      ok: !!client,
      version: SHEET_DB_VERSION,
      timestamp: Util.nowIso(),
      details: {
        clientReady: true,
        spreadsheetMode: client._config && client._config.spreadsheetId ? 'by_id' : 'active'
      }
    };
```

```
  } catch (err) {
    return {
      ok: false,
      version: SHEET_DB_VERSION,
      timestamp: new Date().toISOString(),
      details: {
        code: err.code || 'HEALTH_CHECK_FAILED',
        message: err.message || 'Unknown error.'
      }
    };
  }
}

/**
 * Creates a client using Script Properties with optional explicit overrides.
 * Precedence: explicit options > script properties > defaults.
 * @param {Object=} options
 * @return {DbClient}
 */
function _sheetDbCreateClientFromScriptProperties(options) {
  var merged = mergeClientOptionsWithScriptProperties(options || {});
  return _sheetDbCreateClient(merged);
}

/**
 * Public Apps Script library namespace.
 *
 * Consumers should call library exports through this object:
 * `LibraryIdentifier.SheetDb.createClient(...)`.
 *
 * @type {{
 *   createClient: function(Object=): DbClient,
 *   createClientFromScriptProperties: function(Object=): DbClient,
 *   bootstrap: function(Object=): DbClient,
 *   healthCheck: function(): Object,
 *   version: string
 * }}
 */
var SheetDb = {
  createClient: function(options) {
    return _sheetDbCreateClient(options);
  },

  createClientFromScriptProperties: function(options) {
    return _sheetDbCreateClientFromScriptProperties(options);
  },

  bootstrap: function(options) {
    return _sheetDbBootstrap(options);
  },

  healthCheck: function() {
    return _sheetDbHealthCheck();
  },

  version: SHEET_DB_VERSION
};

/**
 * Backward-compatible factory export.
 * @deprecated Use `SheetDb.createClient` instead.
 * @param {Object=} options
 * @return {DbClient}
 */
function createSheetDbClient(options) {
  return SheetDb.createClient(options);
```

```
}
```
'''

01_DbClient.gs

Source path: 'sheet_db_lib/01_DbClient.gs'

'''javascript
```
/**
 * Main entrypoint for interacting with sheet-backed tables.
 */
class DbClient {
  /**
   * @param {Config=} config
   */
  constructor(config) {
    this._config = config || new Config();
    this._schemas = new SchemaRegistry();
    this._sheets = new SheetsGateway(this._config);
    this._audit = new AuditLogger();
    this._tx = new TransactionManager(this._config.lockTimeoutMs);
  }

  /**
   * Registers a table schema.
   * @param {string} tableName
   * @param {Object} schema
   * @return {DbClient}
   */
  registerTable(tableName, schema) {
    this._schemas.register(tableName, schema);
    this._sheets.ensureTable(tableName, schema.columns);
    return this;
  }

  /**
   * Returns a table repository.
   * @param {string} tableName
   * @return {TableRepository}
   */
  table(tableName) {
    this._schemas.get(tableName);
    return new TableRepository(
      tableName,
      this._config,
      this._schemas,
      this._sheets,
      this._audit
    );
  }

  /**
   * Gets or registers a table schema.
   *
   * - When 'schema' is provided, registers/overwrites the schema and ensures the table.
   * - When 'schema' is omitted, returns the current schema.
   *
   * @param {string} tableName
   * @param {Object=} schema
   * @return {Object|DbClient}
   */
  schema(tableName, schema) {
    if (schema) {
      this.registerTable(tableName, schema);
```

```
      return this;
    }
    return this._schemas.get(tableName);
  }

  /**
   * Writes an audit entry through the configured audit logger.
   * @param {string} action
   * @param {string} tableName
   * @param {Object=} metadata
   */
  audit(action, tableName, metadata) {
    this._audit.log(action, tableName, metadata || {});
  }

  /**
   * Runs a set of operations under lock.
   * @template T
   * @param {function(DbClient): T} callback
   * @return {T}
   */
  transaction(callback) {
    var self = this;
    return this._tx.run(function() {
      return callback(self);
    });
  }
}

'''


# 02_Config.gs

**Source path:** 'sheet_db_lib/02_Config.gs'

'''javascript
/**
 * Immutable runtime configuration for the Sheet DB client.
 */
class Config {
  /**
   * @param {Object=} options
   * @param {string=} options.spreadsheetId Spreadsheet ID; defaults to active spreadsheet.
   * @param {string=} options.idColumn Record primary key column name.
   * @param {string=} options.createdAtColumn Created timestamp column name.
   * @param {string=} options.updatedAtColumn Updated timestamp column name.
   * @param {string=} options.deletedAtColumn Soft-delete timestamp column name.
   * @param {number=} options.lockTimeoutMs Lock acquisition timeout for transactions.
   */
  constructor(options) {
    var opts = options || {};
    this.spreadsheetId = opts.spreadsheetId || '';
    this.idColumn = opts.idColumn || 'id';
    this.createdAtColumn = opts.createdAtColumn || 'createdAt';
    this.updatedAtColumn = opts.updatedAtColumn || 'updatedAt';
    this.deletedAtColumn = opts.deletedAtColumn || 'deletedAt';
    this.lockTimeoutMs = Util.isNil(opts.lockTimeoutMs) ? 30000 : Number(opts.lockTimeoutMs);
    this.validate();
    Object.freeze(this);
  }

  /**
   * @return {void}
   */
  validate() {
```

```
    var columns = [
      this.idColumn,
      this.createdAtColumn,
      this.updatedAtColumn,
      this.deletedAtColumn
    ];
    var unique = {};
    columns.forEach(function(column) {
      if (!column || typeof column !== 'string') {
        throw new ConfigError('System column names must be non-empty strings.');
      }
      if (unique[column]) {
        throw new ConfigError('System column names must be unique. Duplicate: ' + column);
      }
      unique[column] = true;
    });

    if (isNaN(this.lockTimeoutMs) || this.lockTimeoutMs <= 0) {
      throw new ConfigError('lockTimeoutMs must be a positive number.');
    }
  }
}

'''
```

03_Errors.gs

Source path: `sheet_db_lib/03_Errors.gs`

```javascript
/**
 * Base error for the Sheet DB library.
 */
class SheetDbError extends Error {
  /**
   * @param {string} message Human-readable error message.
   * @param {string=} code Stable error code for programmatic handling.
   */
  constructor(message, code) {
    super(message);
    this.name = this.constructor.name;
    this.code = code || 'SHEET_DB_ERROR';
  }
}

/**
 * Raised when configuration is invalid or missing.
 */
class ConfigError extends SheetDbError {
  /**
   * @param {string} message
   */
  constructor(message) {
    super(message, 'CONFIG_ERROR');
  }
}

/**
 * Raised when schema definitions are invalid.
 */
class SchemaError extends SheetDbError {
  /**
   * @param {string} message
   */
  constructor(message) {
```

```
    super(message, 'SCHEMA_ERROR');
  }
}

/**
 * Raised when records fail validation.
 */
class ValidationError extends SheetDbError {
  /**
   * @param {string} message
   */
  constructor(message) {
    super(message, 'VALIDATION_ERROR');
  }
}

/**
 * Raised when underlying sheet operations fail.
 */
class StorageError extends SheetDbError {
  /**
   * @param {string} message
   */
  constructor(message) {
    super(message, 'STORAGE_ERROR');
  }
}

/**
 * Raised for optimistic transaction failures.
 */
class TransactionError extends SheetDbError {
  /**
   * @param {string} message
   */
  constructor(message) {
    super(message, 'TRANSACTION_ERROR');
  }
}

'''


# 04_Util.gs

**Source path:** 'sheet_db_lib/04_Util.gs'

'''javascript
/**
 * Utility helpers for Sheet DB.
 */
class Util {
  /**
   * @return {string} RFC3339 timestamp in UTC.
   */
  static nowIso() {
    return new Date().toISOString();
  }

  /**
   * Creates a shallow clone of an object.
   * @param {Object} source
   * @return {Object}
   */
  static clone(source) {
    return Object.assign({}, source || {});
```

```
}

/**
 * Creates a stable row ID with a prefix.
 * @param {string=} prefix
 * @return {string}
 */
static generateId(prefix) {
  var p = prefix || 'row';
  var random = Math.random().toString(36).slice(2, 10);
  return p + '_' + new Date().getTime() + '_' + random;
}

/**
 * Returns true when value is null or undefined.
 * @param {*} value
 * @return {boolean}
 */
static isNil(value) {
  return value === null || value === undefined;
}

/**
 * Returns true when value is empty string, null, or undefined.
 * @param {*} value
 * @return {boolean}
 */
static isBlank(value) {
  return value === '' || Util.isNil(value);
}

/**
 * Throws when a required condition is not met.
 * @param {boolean} condition
 * @param {string} message
 */
static assert(condition, message) {
  if (!condition) {
    throw new ValidationError(message);
  }
}

/**
 * Converts a sheet row to an object using the header order.
 * @param {string[]} headers
 * @param {Array<*>} row
 * @return {Object}
 */
static rowToObject(headers, row) {
  var out = {};
  for (var i = 0; i < headers.length; i++) {
    out[headers[i]] = row[i];
  }
  return out;
}

/**
 * Converts an object to row array by header order.
 * @param {string[]} headers
 * @param {Object} obj
 * @return {Array<*>}
 */
static objectToRow(headers, obj) {
  return headers.map(function(h) {
    return obj[h];
  });
```

```
  }
}

'''


# 05_SchemaRegistry.gs

**Source path:** 'sheet_db_lib/05_SchemaRegistry.gs'

'''javascript
/**
 * Holds table schemas and applies field defaults/validation.
 */
class SchemaRegistry {
  /**
   * Creates a new in-memory schema registry.
   */
  constructor() {
    this._schemas = {};
  }

  /**
   * Registers a table schema.
   * @param {string} tableName
   * @param {Object} schema
   * @param {Array<string>} schema.columns Ordered columns for persistence.
   * @param {Object<string, Object>=} schema.fields Optional field constraints.
   * @return {SchemaRegistry}
   */
  register(tableName, schema) {
    if (!tableName) {
      throw new SchemaError('Table name is required.');
    }
    if (!schema || !Array.isArray(schema.columns) || schema.columns.length === 0) {
      throw new SchemaError('Schema for ' + tableName + ' must provide non-empty columns array.');
    }
    var seen = {};
    schema.columns.forEach(function(column) {
      if (!column || typeof column !== 'string') {
        throw new SchemaError('All columns must be non-empty strings for table: ' + tableName);
      }
      if (seen[column]) {
        throw new SchemaError('Duplicate column "' + column + '" in table: ' + tableName);
      }
      seen[column] = true;
    });

    var fields = Util.clone(schema.fields || {});
    Object.keys(fields).forEach(function(fieldName) {
      if (!seen[fieldName]) {
        throw new SchemaError(
          'Field rules defined for unknown column "' + fieldName + '" in table: ' + tableName
        );
      }
    });

    this._schemas[tableName] = {
      columns: schema.columns.slice(),
      fields: fields
    };
    return this;
  }

  /**
   * Returns schema for a table.
```

```
   * @param {string} tableName
   * @return {Object}
   */
  get(tableName) {
    var schema = this._schemas[tableName];
    if (!schema) {
      throw new SchemaError('No schema registered for table: ' + tableName);
    }
    return {
      columns: schema.columns.slice(),
      fields: Util.clone(schema.fields)
    };
  }

  /**
   * Validates and normalizes a record.
   * @param {string} tableName
   * @param {Object} record
   * @param {boolean=} isPartial True for patch-like updates.
   * @return {Object}
   */
  normalizeRecord(tableName, record, isPartial) {
    var schema = this.get(tableName);
    var out = Util.clone(record || {});
    schema.columns.forEach(function(column) {
      var fieldRules = schema.fields[column] || {};
      var hasValue = !Util.isNil(out[column]);

      if (!hasValue && fieldRules.hasOwnProperty('default')) {
        var defaultValue = fieldRules.default;
        out[column] = typeof defaultValue === 'function' ? defaultValue() : defaultValue;
      }

      if (!isPartial && fieldRules.required && Util.isNil(out[column])) {
        throw new ValidationError('Field "' + column + '" is required.');
      }

      if (!Util.isNil(out[column]) && fieldRules.type) {
        var expected = fieldRules.type;
        var actual = Object.prototype.toString.call(out[column]).slice(8, -1).toLowerCase();
        if (actual !== expected.toLowerCase()) {
          throw new ValidationError(
            'Field "' + column + '" expected type ' + expected + ' but got ' + actual + '.'
          );
        }
      }
    });

    return out;
  }
}

'''


# 06_SheetsGateway.gs

**Source path:** `sheet_db_lib/06_SheetsGateway.gs`

'''javascript
/**
 * Thin wrapper around SpreadsheetApp APIs.
 */
class SheetsGateway {
  /**
   * @param {Config} config
```

```
 */
constructor(config) {
  this._config = config;
}

/**
 * @return {GoogleAppsScript.Spreadsheet.Spreadsheet}
 */
getSpreadsheet() {
  if (this._config.spreadsheetId) {
    return SpreadsheetApp.openById(this._config.spreadsheetId);
  }
  return SpreadsheetApp.getActiveSpreadsheet();
}

/**
 * @param {string} tableName
 * @param {boolean=} createWhenMissing
 * @return {GoogleAppsScript.Spreadsheet.Sheet}
 */
getSheet(tableName, createWhenMissing) {
  var ss = this.getSpreadsheet();
  var sheet = ss.getSheetByName(tableName);
  if (!sheet && createWhenMissing) {
    sheet = ss.insertSheet(tableName);
  }
  if (!sheet) {
    throw new StorageError('Sheet not found for table: ' + tableName);
  }
  return sheet;
}

/**
 * @param {string} tableName
 * @return {{headers: string[], rows: Array<Array<*>>}}
 */
readTable(tableName) {
  var sheet = this.getSheet(tableName, false);
  var range = sheet.getDataRange();
  var values = range.getValues();
  if (values.length === 0) {
    return { headers: [], rows: [] };
  }
  return {
    headers: values[0],
    rows: values.slice(1)
  };
}

/**
 * Ensures table exists with given headers.
 * @param {string} tableName
 * @param {string[]} headers
 */
ensureTable(tableName, headers) {
  var sheet = this.getSheet(tableName, true);
  var lastRow = sheet.getLastRow();
  if (lastRow === 0) {
    sheet.getRange(1, 1, 1, headers.length).setValues([headers]);
    return;
  }

  var existingHeaders = sheet.getRange(1, 1, 1, sheet.getLastColumn()).getValues()[0]
    .map(function(value) { return String(value); });
  var expectedHeaders = headers.map(function(value) { return String(value); });
  if (existingHeaders.join('|') !== expectedHeaders.join('|')) {
```

```
      throw new StorageError('Header mismatch for table: ' + tableName);
    }
  }

  /**
   * Writes all rows for a table, replacing existing body rows.
   * @param {string} tableName
   * @param {string[]} headers
   * @param {Array<Array<*>>} rows
   */
  writeAllRows(tableName, headers, rows) {
    var sheet = this.getSheet(tableName, true);
    this.ensureTable(tableName, headers);

    var maxRows = Math.max(sheet.getMaxRows(), rows.length + 1);
    if (sheet.getMaxRows() < maxRows) {
      sheet.insertRowsAfter(sheet.getMaxRows(), maxRows - sheet.getMaxRows());
    }

    if (sheet.getLastRow() > 1) {
      sheet.getRange(2, 1, sheet.getLastRow() - 1, headers.length).clearContent();
    }

    if (rows.length > 0) {
      sheet.getRange(2, 1, rows.length, headers.length).setValues(rows);
    }
  }
}

'''


# 07_TableRepository.gs

**Source path:** 'sheet_db_lib/07_TableRepository.gs'

'''javascript
/**
 * CRUD repository for a specific table.
 */
class TableRepository {
  /**
   * @param {string} tableName
   * @param {Config} config
   * @param {SchemaRegistry} schemaRegistry
   * @param {SheetsGateway} sheetsGateway
   * @param {AuditLogger} auditLogger
   */
  constructor(tableName, config, schemaRegistry, sheetsGateway, auditLogger) {
    this._tableName = tableName;
    this._config = config;
    this._schemas = schemaRegistry;
    this._sheets = sheetsGateway;
    this._audit = auditLogger;
  }

  /**
   * @return {Array<Object>}
   */
  findAll() {
    var schema = this._schemas.get(this._tableName);
    this._sheets.ensureTable(this._tableName, schema.columns);
    var data = this._sheets.readTable(this._tableName);
    return data.rows
      .map(function(row) { return Util.rowToObject(data.headers, row); })
      .filter(function(record) { return Util.isBlank(record[this._config.deletedAtColumn]); }, this);
```

```
}

/**
 * @param {string} id
 * @return {Object|null}
 */
findById(id) {
  var rows = this.findAll();
  for (var i = 0; i < rows.length; i++) {
    if (rows[i][this._config.idColumn] === id) {
      return rows[i];
    }
  }
  return null;
}

/**
 * @param {Object} payload
 * @return {Object}
 */
insert(payload) {
  var schema = this._schemas.get(this._tableName);
  this._sheets.ensureTable(this._tableName, schema.columns);
  var record = this._schemas.normalizeRecord(this._tableName, payload, false);
  var now = Util.nowIso();
  record[this._config.idColumn] = record[this._config.idColumn] || Util.generateId(this._tableName);
  record[this._config.createdAtColumn] = record[this._config.createdAtColumn] || now;
  record[this._config.updatedAtColumn] = now;
  if (!record.hasOwnProperty(this._config.deletedAtColumn)) {
    record[this._config.deletedAtColumn] = '';
  }

  var tableData = this._sheets.readTable(this._tableName);
  var existingRows = tableData.rows.map(function(row) { return Util.rowToObject(tableData.headers, row);
  var idColumn = this._config.idColumn;
  var duplicate = existingRows.some(function(row) {
    return row[idColumn] === record[idColumn];
  });
  if (duplicate) {
    throw new ValidationError('Record ID already exists: ' + record[idColumn]);
  }
  existingRows.push(record);
  var rowValues = existingRows.map(function(r) { return Util.objectToRow(schema.columns, r); });
  this._sheets.writeAllRows(this._tableName, schema.columns, rowValues);
  this._audit.log('insert', this._tableName, { id: record[this._config.idColumn] });
  return record;
}

/**
 * @param {string} id
 * @param {Object} patch
 * @return {Object}
 */
update(id, patch) {
  var schema = this._schemas.get(this._tableName);
  var normalizedPatch = this._schemas.normalizeRecord(this._tableName, patch, true);
  var tableData = this._sheets.readTable(this._tableName);
  var records = tableData.rows.map(function(row) { return Util.rowToObject(tableData.headers, row); });
  var found = null;

  for (var i = 0; i < records.length; i++) {
    if (records[i][this._config.idColumn] === id) {
      found = records[i];
      Object.keys(normalizedPatch).forEach(function(key) {
        if (key !== this._config.idColumn && key !== this._config.createdAtColumn) {
          found[key] = normalizedPatch[key];
```

```
          }
        }, this);
        found[this._config.updatedAtColumn] = Util.nowIso();
        break;
      }
    }

    if (!found) {
      throw new StorageError('Record not found for id: ' + id);
    }

    var rowValues = records.map(function(r) { return Util.objectToRow(schema.columns, r); });
    this._sheets.writeAllRows(this._tableName, schema.columns, rowValues);
    this._audit.log('update', this._tableName, { id: id, fields: Object.keys(normalizedPatch) });
    return found;
  }

  /**
   * Soft-deletes a record.
   * @param {string} id
   * @return {Object}
   */
  remove(id) {
    var result = this.update(id, (function(col, now) {
      var patch = {};
      patch[col] = now;
      return patch;
    })(this._config.deletedAtColumn, Util.nowIso()));
    this._audit.log('remove', this._tableName, { id: id });
    return result;
  }
}

'''
```

08_TransactionManager.gs

Source path: 'sheet_db_lib/08_TransactionManager.gs'

```javascript
/**
 * Runs operations under ScriptLock for coarse transaction semantics.
 */
class TransactionManager {
  /**
   * @param {number=} lockTimeoutMs
   */
  constructor(lockTimeoutMs) {
    this._lockTimeoutMs = lockTimeoutMs || 30000;
  }

  /**
   * Executes callback while holding a script lock.
   * @template T
   * @param {function(): T} callback
   * @return {T}
   */
  run(callback) {
    var lock = LockService.getScriptLock();
    if (!lock.tryLock(this._lockTimeoutMs)) {
      throw new TransactionError('Failed to acquire script lock within timeout.');
    }
    try {
      return callback();
    } finally {
```

```
      lock.releaseLock();
    }
  }
}
```

```

# 09_AuditLogger.gs

**Source path:** `sheet_db_lib/09_AuditLogger.gs`

```javascript
/**
 * Minimal pluggable audit logger.
 */
class AuditLogger {
  /**
   * @param {Object=} options
   * @param {Function=} options.sink Optional custom sink function.
   */
  constructor(options) {
    var opts = options || {};
    this._sink = opts.sink || function(entry) {
      Logger.log(JSON.stringify(entry));
    };
  }

  /**
   * Emits an audit entry.
   * @param {string} action
   * @param {string} tableName
   * @param {Object=} metadata
   */
  log(action, tableName, metadata) {
    this._sink({
      at: Util.nowIso(),
      action: action,
      table: tableName,
      metadata: Util.clone(metadata || {})
    });
  }
}
```

```

# 10_ScriptProperties.gs

**Source path:** `sheet_db_lib/10_ScriptProperties.gs`

```javascript
/**
 * Script Properties helper for secure and environment-specific configuration.
 */
class ScriptPropertiesHelper {
  /**
   * Creates a helper bound to Script Properties store.
   */
  constructor() {
    this._props = PropertiesService.getScriptProperties();
  }

  /**
   * Returns a property value, or a default when missing.
   * @param {string} key
```

```
 * @param {*=} defaultValue
 * @return {*}
 */
get(key, defaultValue) {
  var value = this._props.getProperty(key);
  return value === null ? defaultValue : value;
}

/**
 * Returns a required property value or throws when missing/blank.
 * @param {string} key
 * @return {string}
 */
getRequired(key) {
  var value = this.get(key, '');
  if (value === '') {
    throw new ConfigError('Missing required script property: ' + key);
  }
  return String(value);
}

/**
 * Returns a boolean property value.
 * Accepted true values: true, 1, yes, y, on.
 * Accepted false values: false, 0, no, n, off.
 * @param {string} key
 * @param {boolean=} defaultValue
 * @return {boolean}
 */
getBoolean(key, defaultValue) {
  var raw = this.get(key, null);
  if (raw === null || raw === '') {
    return defaultValue === undefined ? false : !!defaultValue;
  }
  var normalized = String(raw).trim().toLowerCase();
  if (['true', '1', 'yes', 'y', 'on'].indexOf(normalized) !== -1) {
    return true;
  }
  if (['false', '0', 'no', 'n', 'off'].indexOf(normalized) !== -1) {
    return false;
  }
  throw new ConfigError('Script property is not a valid boolean: ' + key);
}

/**
 * Returns a numeric property value.
 * @param {string} key
 * @param {number=} defaultValue
 * @return {number}
 */
getNumber(key, defaultValue) {
  var raw = this.get(key, null);
  if (raw === null || raw === '') {
    return defaultValue === undefined ? 0 : Number(defaultValue);
  }
  var n = Number(raw);
  if (isNaN(n)) {
    throw new ConfigError('Script property is not a valid number: ' + key);
  }
  return n;
}

/**
 * Returns all script properties as key-value object.
 * @return {Object<string, string>}
 */
```

```
getAll() {
  return this._props.getProperties();
}

/**
 * Returns all script properties with sensitive values redacted.
 * @return {Object<string, string>}
 */
getAllRedacted() {
  var all = this.getAll();
  var out = {};
  Object.keys(all).forEach(function(key) {
    out[key] = ScriptPropertiesHelper.shouldRedactKey(key) ? '[REDACTED]' : all[key];
  });
  return out;
}

/**
 * Sets a script property value.
 * @param {string} key
 * @param {*} value
 */
set(key, value) {
  this._props.setProperty(key, String(value));
}

/**
 * Sets multiple script properties.
 * @param {Object<string, *>} obj
 */
setMany(obj) {
  var stringified = {};
  Object.keys(obj || {}).forEach(function(key) {
    stringified[key] = String(obj[key]);
  });
  this._props.setProperties(stringified, false);
}

/**
 * Deletes a script property.
 * @param {string} key
 */
delete(key) {
  this._props.deleteProperty(key);
}

/**
 * Returns true when a script property exists and is non-empty.
 * @param {string} key
 * @return {boolean}
 */
has(key) {
  var value = this._props.getProperty(key);
  return value !== null && value !== '';
}

/**
 * Returns true when key should be redacted in debug output.
 * @param {string} key
 * @return {boolean}
 */
static shouldRedactKey(key) {
  var upper = String(key || '').toUpperCase();
  if (upper === 'APP_PASSCODE' || upper === 'SIGNING_SECRET' || upper === 'WEBHOOK_SECRET' || upper === '
    return true;
  }
```

```
    return upper.indexOf('TOKEN') !== -1 || upper.indexOf('SECRET') !== -1 || upper.indexOf('PASSWORD') !==
  }
}

/**
 * Reads SheetDb client-related config from Script Properties.
 *
 * Supported keys:
 * - SHEET_DB_SPREADSHEET_ID
 * - SHEET_DB_ID_COLUMN
 * - SHEET_DB_CREATED_AT_COLUMN
 * - SHEET_DB_UPDATED_AT_COLUMN
 * - SHEET_DB_DELETED_AT_COLUMN
 * - SHEET_DB_LOCK_TIMEOUT_MS
 *
 * Also reserved for host-layer secure use:
 * - APP_PASSCODE
 * - SIGNING_SECRET
 * - WEBHOOK_SECRET
 * - API_KEY
 *
 * @return {Object}
 */
function readSheetDbConfigFromScriptProperties() {
  var helper = new ScriptPropertiesHelper();
  var config = {};

  if (helper.has('SHEET_DB_SPREADSHEET_ID')) {
    config.spreadsheetId = helper.get('SHEET_DB_SPREADSHEET_ID', '');
  }
  if (helper.has('SHEET_DB_ID_COLUMN')) {
    config.idColumn = helper.get('SHEET_DB_ID_COLUMN', 'id');
  }
  if (helper.has('SHEET_DB_CREATED_AT_COLUMN')) {
    config.createdAtColumn = helper.get('SHEET_DB_CREATED_AT_COLUMN', 'createdAt');
  }
  if (helper.has('SHEET_DB_UPDATED_AT_COLUMN')) {
    config.updatedAtColumn = helper.get('SHEET_DB_UPDATED_AT_COLUMN', 'updatedAt');
  }
  if (helper.has('SHEET_DB_DELETED_AT_COLUMN')) {
    config.deletedAtColumn = helper.get('SHEET_DB_DELETED_AT_COLUMN', 'deletedAt');
  }
  if (helper.has('SHEET_DB_LOCK_TIMEOUT_MS')) {
    config.lockTimeoutMs = helper.getNumber('SHEET_DB_LOCK_TIMEOUT_MS', 30000);
  }

  return config;
}

/**
 * Merges explicit client options with script properties-based config.
 * Precedence: explicit options > script properties > hardcoded defaults.
 *
 * @param {Object=} options
 * @return {Object}
 */
function mergeClientOptionsWithScriptProperties(options) {
  var fromProps = readSheetDbConfigFromScriptProperties();
  var explicit = options || {};
  var merged = {};
  Object.keys(fromProps).forEach(function(key) {
    merged[key] = fromProps[key];
  });
  Object.keys(explicit).forEach(function(key) {
    merged[key] = explicit[key];
  });
```

```
  return merged;
}

/**
 * Example setup helper for consumers (manual one-time setup).
 *
 * Example:
 * PropertiesService.getScriptProperties().setProperty('SHEET_DB_SPREADSHEET_ID', '...');
 */
function exampleSetSheetDbScriptProperties() {
  // Intentionally no-op; serves as in-library usage documentation only.
}
```

```

# 00_WebAppEntry.gs

**Source path:** `slack_host/00_WebAppEntry.gs`

```javascript
/**
 * Slack host entrypoint (web app).
 */

/**
 * Creates a DB client from SheetDb library and registers host schemas.
 * @return {DbClient}
 */
function createHostDbClient() {
  var db = SheetDb.createClientFromScriptProperties();

  db.schema('students', {
    columns: ['id', 'name', 'source', 'createdAt', 'updatedAt', 'deletedAt'],
    fields: {
      id: { required: true, type: 'string' },
      name: { required: true, type: 'string' },
      source: { default: 'slack', type: 'string' }
    }
  });

  db.schema('enrollments', {
    columns: ['id', 'studentId', 'courseId', 'status', 'source', 'createdAt', 'updatedAt', 'deletedAt'],
    fields: {
      studentId: { required: true, type: 'string' },
      courseId: { required: true, type: 'string' },
      status: { default: 'enrolled', type: 'string' },
      source: { default: 'slack', type: 'string' }
    }
  });

  db.schema('automations', {
    columns: ['id', 'requestId', 'workflow', 'eventType', 'state', 'createdAt', 'updatedAt', 'deletedAt'],
    fields: {
      requestId: { required: true, type: 'string' },
      workflow: { required: true, type: 'string' },
      eventType: { required: true, type: 'string' },
      state: { default: 'received', type: 'string' }
    }
  });

  db.schema('audit_logs', {
    columns: ['id', 'requestId', 'source', 'action', 'status', 'message', 'metadata', 'createdAt', 'updated
    fields: {
      requestId: { required: true, type: 'string' },
      source: { required: true, type: 'string' },
```
```

```
      action: { required: true, type: 'string' },
      status: { required: true, type: 'string' },
      message: { type: 'string' },
      metadata: { type: 'string' }
    }
  });

  return db;
}

/**
 * Creates the host-layer router with all dependencies wired.
 * @param {DbClient} db
 * @param {HostConfig} hostConfig
 * @return {SlackRouter}
 */
function createHostRouter(db, hostConfig) {
  var parser = new SlackPayloadParser();
  var security = new SlackSecurity(hostConfig);
  var contexts = new RequestContextFactory(hostConfig);
  var formatter = new ResultFormatter();

  var enrollmentService = new LmsEnrollmentService(db);
  var automationService = new LmsAutomationService(db);
  var workflowHandlers = new WorkflowWebhookHandlers(enrollmentService, automationService);
  var slackService = new SlackService(enrollmentService, workflowHandlers, formatter);

  return new SlackRouter(parser, security, contexts, slackService, formatter, db);
}

/**
 * Webhook entrypoint.
 * @param {GoogleAppsScript.Events.DoPost} e
 * @return {GoogleAppsScript.Content.TextOutput}
 */
function doPost(e) {
  var db = createHostDbClient();
  var hostConfig = new HostConfig(new ScriptPropertiesHelper());
  var router = createHostRouter(db, hostConfig);
  var result = router.handle(e);

  return ContentService
    .createTextOutput(JSON.stringify(result.slack || result))
    .setMimeType(ContentService.MimeType.JSON);
}
```

```

# 01_SlackRouter.gs

**Source path:** `slack_host/01_SlackRouter.gs`

```javascript
/**
 * HTTP router boundary: parse -> verify -> context -> app service.
 */
class SlackRouter {
  /**
   * @param {SlackPayloadParser} parser
   * @param {SlackSecurity} security
   * @param {RequestContextFactory} contextFactory
   * @param {SlackService} slackService
   * @param {ResultFormatter} resultFormatter
   * @param {DbClient} db
   */
```
```

```
  constructor(parser, security, contextFactory, slackService, resultFormatter, db) {
    this._parser = parser;
    this._security = security;
    this._contexts = contextFactory;
    this._service = slackService;
    this._results = resultFormatter;
    this._db = db;
  }

  /**
   * @param {GoogleAppsScript.Events.DoPost} e
   * @return {Object}
   */
  handle(e) {
    var parsed = this._parser.parseEvent(e);
    var context = this._contexts.create(parsed.routeType, parsed.payload, parsed.envelope);

    var auth = this._security.verify(parsed);
    if (!auth.ok) {
      var denied = this._results.failure(auth.code, auth.message, context, {});
      this._writeAudit(context, denied);
      return denied;
    }

    var result;
    try {
      if (parsed.routeType === 'slash_command') {
        result = this._service.handleSlashCommand(context, parsed.payload);
      } else if (parsed.routeType === 'workflow_webhook') {
        result = this._service.handleWorkflowWebhook(context, parsed.payload);
      } else {
        result = this._results.failure('UNSUPPORTED_PAYLOAD', 'Unsupported payload.', context, {});
      }
    } catch (err) {
      result = this._results.failure(err.code || 'HOST_ERROR', err.message || 'Unexpected error.', context,
    }

    this._writeAudit(context, result);
    return result;
  }

  /** @private */
  _writeAudit(context, result) {
    this._db.table('audit_logs').insert({
      requestId: context.requestId,
      source: context.source,
      action: context.routeType,
      status: result.ok ? 'success' : 'failure',
      message: result.message,
      metadata: JSON.stringify({ code: result.code, requestId: result.requestId })
    });
  }
}

'''


# 02_SlackPayloadParser.gs

**Source path:** 'slack_host/02_SlackPayloadParser.gs'

'''javascript
/**
 * Parses and normalizes inbound Slack-related payloads.
 */
class SlackPayloadParser {
```

```
/**
 * @param {GoogleAppsScript.Events.DoPost} e
 * @return {{routeType: string, payload: Object, envelope: Object}}
 */
parseEvent(e) {
  var rawBody = (e && e.postData && e.postData.contents) ? e.postData.contents : '';
  var params = this._parseFormEncoded(rawBody);

  if (params.command) {
    return {
      routeType: 'slash_command',
      payload: this._normalizeSlashCommand(params),
      envelope: { rawBody: rawBody, params: params }
    };
  }

  var jsonPayload = this._parseJson(rawBody);
  if (jsonPayload) {
    return {
      routeType: 'workflow_webhook',
      payload: this._normalizeWorkflowWebhook(jsonPayload),
      envelope: { rawBody: rawBody, json: jsonPayload }
    };
  }

  return { routeType: 'unknown', payload: {}, envelope: { rawBody: rawBody } };
}

/** @private */
_normalizeSlashCommand(payload) {
  return {
    command: String(payload.command || '').trim(),
    text: String(payload.text || '').trim(),
    teamId: String(payload.team_id || ''),
    channelId: String(payload.channel_id || ''),
    userId: String(payload.user_id || ''),
    triggerId: String(payload.trigger_id || ''),
    responseUrl: String(payload.response_url || ''),
    requestId: String(payload.request_id || Util.generateId('slash'))
  };
}

/** @private */
_normalizeWorkflowWebhook(payload) {
  var data = payload.data || payload;
  return {
    workflow: String(payload.workflow || payload.workflow_name || 'unknown_workflow'),
    eventType: String(payload.eventType || payload.event_type || 'workflow_event'),
    requestId: String(payload.requestId || payload.request_id || Util.generateId('wf')),
    studentId: String(data.studentId || data.student_id || ''),
    courseId: String(data.courseId || data.course_id || ''),
    actorId: String(data.actorId || data.actor_id || ''),
    status: String(data.status || '')
  };
}

/** @private */
_parseJson(raw) {
  try {
    if (!raw) return null;
    return JSON.parse(raw);
  } catch (err) {
    return null;
  }
}
```

```
  /** @private */
  _parseFormEncoded(body) {
    var out = {};
    if (!body) return out;
    body.split('&').forEach(function(pair) {
      var tokens = pair.split('=');
      var key = decodeURIComponent(tokens[0] || '').replace(/\+/g, ' ');
      var rawValue = tokens.length > 1 ? tokens.slice(1).join('=') : '';
      var value = decodeURIComponent(rawValue).replace(/\+/g, ' ');
      if (key) out[key] = value;
    });
    return out;
  }
}

'''
```

```
# 03_SlackService.gs

**Source path:** 'slack_host/03_SlackService.gs'

'''javascript
/**
 * Application service that orchestrates Slack routes into LMS services.
 */
class SlackService {
  /**
   * @param {LmsEnrollmentService} enrollmentService
   * @param {WorkflowWebhookHandlers} workflowHandlers
   * @param {ResultFormatter} resultFormatter
   */
  constructor(enrollmentService, workflowHandlers, resultFormatter) {
    this._enrollment = enrollmentService;
    this._workflow = workflowHandlers;
    this._results = resultFormatter;
  }

  /**
   * @param {Object} context
   * @param {Object} payload
   * @return {Object}
   */
  handleSlashCommand(context, payload) {
    if (payload.command === '/enroll-student') {
      var result = this._enrollment.enrollFromSlash(context, payload);
      return result.ok ? this._results.success(result.message, result.data, context)
        : this._results.failure(result.code, result.message, context, result.data);
    }

    return this._results.failure('UNSUPPORTED_COMMAND', 'Unsupported command: ' + payload.command, context,
  }

  /**
   * @param {Object} context
   * @param {Object} payload
   * @return {Object}
   */
  handleWorkflowWebhook(context, payload) {
    var result = this._workflow.handle(context, payload);
    return result.ok ? this._results.success(result.message, result.data, context)
      : this._results.failure(result.code, result.message, context, result.data);
  }
}

'''
```

04_SlackSecurity.gs

Source path: ‘slack_host/04_SlackSecurity.gs‘

‘‘‘javascript
/**
 * Host-layer security checks for Slack/webhook calls.
 */
class SlackSecurity {
  /**
   * @param {HostConfig} hostConfig
   */
  constructor(hostConfig) {
    this._config = hostConfig;
  }

  /**
   * Best-effort request verification for Apps Script web-app requests.
   * Uses passcode from query/form when available.
   *
   * @param {{routeType: string, payload: Object, envelope: Object}} parsed
   * @return {{ok: boolean, code: string, message: string}}
   */
  verify(parsed) {
    var configured = this._config.getAppPasscode();
    if (!configured) {
      return { ok: true, code: ’NO_PASSCODE_CONFIGURED’, message: ’Passcode check skipped.’ };
    }

    var params = (parsed.envelope && parsed.envelope.params) ? parsed.envelope.params : {};
    var supplied = String(params.passcode || params.api_key || ’’);
    if (supplied && supplied === configured) {
      return { ok: true, code: ’AUTHORIZED’, message: ’Passcode validated.’ };
    }

    return { ok: false, code: ’UNAUTHORIZED’, message: ’Invalid or missing passcode.’ };
  }
}

‘‘‘

05_LmsEnrollmentService.gs

Source path: ‘slack_host/05_LmsEnrollmentService.gs‘

‘‘‘javascript
/**
 * LMS enrollment operations implemented using the SheetDb library only.
 */
class LmsEnrollmentService {
  /**
   * @param {DbClient} db
   */
  constructor(db) {
    this._db = db;
  }

  /**
   * Slash-command flow example:
   * /enroll-student <studentId> <courseId>
   *
   * Upserts into ‘students‘ and ‘enrollments‘, then writes audit log row.
   * @param {Object} context

```
 * @param {Object} payload
 * @return {Object}
 */
enrollFromSlash(context, payload) {
  var args = String(payload.text || '').split(/\s+/).filter(function(v) { return !!v; });
  if (args.length < 2) {
    return { ok: false, code: 'INVALID_ARGUMENTS', message: 'Usage: /enroll-student <studentId> <courseId
  }

  var studentId = args[0];
  var courseId = args[1];
  var out = this._upsertEnrollment(context, studentId, courseId, 'slash_command');
  return out;
}

/**
 * Workflow webhook flow example for enrollment updates/inserts.
 * @param {Object} context
 * @param {Object} payload
 * @return {Object}
 */
upsertFromWorkflow(context, payload) {
  if (!payload.studentId || !payload.courseId) {
    return { ok: false, code: 'INVALID_WORKFLOW_PAYLOAD', message: 'studentId and courseId are required.'
  }

  return this._upsertEnrollment(context, payload.studentId, payload.courseId, 'workflow_webhook', payload
}

/** @private */
_upsertEnrollment(context, studentId, courseId, source, status) {
  var students = this._db.table('students');
  var enrollments = this._db.table('enrollments');
  var audit = this._db.table('audit_logs');
  var finalStatus = status || 'enrolled';
  var enrollmentRecord;

  this._db.transaction(function() {
    var existingStudent = students.findById(studentId);
    if (!existingStudent) {
      students.insert({ id: studentId, name: studentId, source: source });
    }

    var existingEnrollment = enrollments.findAll().filter(function(r) {
      return r.studentId === studentId && r.courseId === courseId;
    })[0];

    if (existingEnrollment) {
      enrollmentRecord = enrollments.update(existingEnrollment.id, {
        status: finalStatus,
        source: source
      });
    } else {
      enrollmentRecord = enrollments.insert({
        studentId: studentId,
        courseId: courseId,
        status: finalStatus,
        source: source
      });
    }

    audit.insert({
      requestId: context.requestId,
      source: context.source,
      action: 'enrollment_upsert',
      status: 'success',
```

```
        message: 'Enrollment upsert completed.',
        metadata: JSON.stringify({ studentId: studentId, courseId: courseId, enrollmentId: enrollmentRecord
      });
    });

    this._db.audit('enrollment_upsert', 'enrollments', {
      requestId: context.requestId,
      studentId: studentId,
      courseId: courseId,
      enrollmentId: enrollmentRecord.id
    });

    return {
      ok: true,
      code: 'ENROLLMENT_UPSERTED',
      message: 'Enrollment upserted successfully.',
      data: {
        enrollmentId: enrollmentRecord.id,
        studentId: studentId,
        courseId: courseId,
        status: enrollmentRecord.status
      }
    };
  }
}

'''
```

06_LmsAutomationService.gs

Source path: 'slack_host/06_LmsAutomationService.gs'

'''javascript
```
/**
 * LMS automation operations that persist workflow state via SheetDb.
 */
class LmsAutomationService {
  /**
   * @param {DbClient} db
   */
  constructor(db) {
    this._db = db;
  }

  /**
   * Inserts/updates an automation row and writes an audit log row.
   * @param {Object} context
   * @param {Object} payload
   * @return {Object}
   */
  upsertAutomation(context, payload) {
    var automations = this._db.table('automations');
    var audit = this._db.table('audit_logs');
    var workflow = payload.workflow || 'unknown_workflow';
    var eventType = payload.eventType || 'workflow_event';
    var state = payload.status || 'received';

    var existing = automations.findAll().filter(function(r) {
      return r.workflow === workflow && r.requestId === payload.requestId;
    })[0];

    var row;
    if (existing) {
      row = automations.update(existing.id, { eventType: eventType, state: state });
    } else {
```

```
      row = automations.insert({ requestId: payload.requestId, workflow: workflow, eventType: eventType, st
    }

    audit.insert({
      requestId: context.requestId,
      source: context.source,
      action: 'automation_upsert',
      status: 'success',
      message: 'Automation row upserted.',
      metadata: JSON.stringify({ automationId: row.id, workflow: workflow, eventType: eventType })
    });

    return {
      ok: true,
      code: 'AUTOMATION_UPSERTED',
      message: 'Automation row upserted.',
      data: { automationId: row.id, workflow: workflow, eventType: eventType, state: row.state }
    };
  }
}

'''


# 07_RequestContextFactory.gs

**Source path:** 'slack_host/07_RequestContextFactory.gs'

'''javascript
/**
 * Creates normalized request-context objects for host handlers.
 */
class RequestContextFactory {
  /**
   * @param {HostConfig} hostConfig
   */
  constructor(hostConfig) {
    this._config = hostConfig;
  }

  /**
   * @param {string} routeType
   * @param {Object} payload
   * @param {Object=} envelope
   * @return {Object}
   */
  create(routeType, payload, envelope) {
    var env = envelope || {};
    return {
      requestId: payload.requestId || Util.generateId('req'),
      receivedAt: Util.nowIso(),
      routeType: routeType,
      source: routeType,
      actorId: payload.userId || payload.actorId || 'unknown',
      teamId: payload.teamId || '',
      channelId: payload.channelId || this._config.getSlackDefaultChannel(),
      raw: env.rawBody || ''
    };
  }
}

'''


# 08_ResultFormatter.gs
```

Source path: ‘slack_host/08_ResultFormatter.gs‘

‘‘‘javascript
/**
 * Formats host operation results into structured Slack-friendly objects.
 */
class ResultFormatter {
  /**
   * @param {string} message
   * @param {Object=} data
   * @param {Object=} context
   * @return {Object}
   */
  success(message, data, context) {
    return {
      ok: true,
      code: ’OK’,
      message: message,
      requestId: (context && context.requestId) || ’’,
      data: data || {},
      slack: {
        response_type: ’ephemeral’,
        text: message
      }
    };
  }

  /**
   * @param {string} code
   * @param {string} message
   * @param {Object=} context
   * @param {Object=} data
   * @return {Object}
   */
  failure(code, message, context, data) {
    return {
      ok: false,
      code: code || ’ERROR’,
      message: message || ’Unknown error.’,
      requestId: (context && context.requestId) || ’’,
      data: data || {},
      slack: {
        response_type: ’ephemeral’,
        text: ’:warning: ’ + (message || ’Unknown error.’)
      }
    };
  }
}

‘‘‘

09_WorkflowWebhookHandlers.gs

Source path: ‘slack_host/09_WorkflowWebhookHandlers.gs‘

‘‘‘javascript
/**
 * Handles normalized workflow webhook events.
 */
class WorkflowWebhookHandlers {
  /**
   * @param {LmsEnrollmentService} enrollmentService
   * @param {LmsAutomationService} automationService
   */
  constructor(enrollmentService, automationService) {

```
    this._enrollment = enrollmentService;
    this._automation = automationService;
  }

  /**
   * @param {Object} context
   * @param {Object} payload
   * @return {Object}
   */
  handle(context, payload) {
    if (payload.eventType === 'enrollment_requested') {
      return this._enrollment.upsertFromWorkflow(context, payload);
    }
    return this._automation.upsertAutomation(context, payload);
  }
}

'''
```

10_HostConfig.gs

Source path: `slack_host/10_HostConfig.gs`

```javascript
/**
 * Host-layer configuration backed by Script Properties.
 */
class HostConfig {
  /**
   * @param {ScriptPropertiesHelper=} helper
   */
  constructor(helper) {
    this._props = helper || new ScriptPropertiesHelper();
  }

  /** @return {string} */
  getAppPasscode() { return this._props.get('APP_PASSCODE', ''); }
  /** @return {string} */
  getSigningSecret() { return this._props.get('SIGNING_SECRET', ''); }
  /** @return {string} */
  getWebhookSecret() { return this._props.get('WEBHOOK_SECRET', ''); }
  /** @return {string} */
  getApiKey() { return this._props.get('API_KEY', ''); }
  /** @return {string} */
  getSlackBotToken() { return this._props.get('SLACK_BOT_TOKEN', ''); }
  /** @return {string} */
  getSlackSigningSecret() { return this._props.get('SLACK_SIGNING_SECRET', this.getSigningSecret()); }
  /** @return {string} */
  getSlackDefaultChannel() { return this._props.get('SLACK_DEFAULT_CHANNEL', ''); }

  /**
   * @return {Object}
   */
  getClientOverrides() {
    return mergeClientOptionsWithScriptProperties({});
  }
}

'''
```

11_HostTestHarness.gs

Source path: `slack_host/11_HostTestHarness.gs`

```javascript
/**
 * Manual Apps Script test harness for the split Slack host architecture.
 */

/**
 * Builds host dependencies for manual tests.
 * Uses TEST_SPREADSHEET_ID script property when available.
 * @return {{db: DbClient, hostConfig: HostConfig, router: SlackRouter}}
 */
function _buildHostTestContext() {
  var props = new ScriptPropertiesHelper();
  var testSpreadsheetId = props.get('TEST_SPREADSHEET_ID', '');
  var options = testSpreadsheetId ? { spreadsheetId: testSpreadsheetId } : {};

  var db = SheetDb.createClientFromScriptProperties(options);

  db.schema('students', {
    columns: ['id', 'name', 'source', 'createdAt', 'updatedAt', 'deletedAt'],
    fields: {
      id: { required: true, type: 'string' },
      name: { required: true, type: 'string' },
      source: { default: 'slack', type: 'string' }
    }
  });

  db.schema('enrollments', {
    columns: ['id', 'studentId', 'courseId', 'status', 'source', 'createdAt', 'updatedAt', 'deletedAt'],
    fields: {
      studentId: { required: true, type: 'string' },
      courseId: { required: true, type: 'string' },
      status: { default: 'enrolled', type: 'string' },
      source: { default: 'slack', type: 'string' }
    }
  });

  db.schema('automations', {
    columns: ['id', 'requestId', 'workflow', 'eventType', 'state', 'createdAt', 'updatedAt', 'deletedAt'],
    fields: {
      requestId: { required: true, type: 'string' },
      workflow: { required: true, type: 'string' },
      eventType: { required: true, type: 'string' },
      state: { default: 'received', type: 'string' }
    }
  });

  db.schema('audit_logs', {
    columns: ['id', 'requestId', 'source', 'action', 'status', 'message', 'metadata', 'createdAt', 'updated
    fields: {
      requestId: { required: true, type: 'string' },
      source: { required: true, type: 'string' },
      action: { required: true, type: 'string' },
      status: { required: true, type: 'string' },
      message: { type: 'string' },
      metadata: { type: 'string' }
    }
  });

  var hostConfig = new HostConfig(props);
  var router = createHostRouter(db, hostConfig);
  return { db: db, hostConfig: hostConfig, router: router };
}

/**
 * Encodes form payload.
 * @param {Object} obj
```

```
 * @return {string}
 */
function _toFormBody(obj) {
  return Object.keys(obj).map(function(key) {
    return encodeURIComponent(key) + '=' + encodeURIComponent(String(obj[key]));
  }).join('&');
}

/**
 * Builds a mock DoPost event object.
 * @param {string} body
 * @return {GoogleAppsScript.Events.DoPost}
 */
function _mockDoPostEvent(body) {
  return {
    postData: {
      contents: body,
      type: 'application/x-www-form-urlencoded'
    }
  };
}

/**
 * testSlashEnrollMock
 * Simulates `/enroll-student` and runs full router path.
 */
function testSlashEnrollMock() {
  var ctx = _buildHostTestContext();
  var suffix = new Date().getTime();
  var passcode = ctx.hostConfig.getAppPasscode();
  var payload = {
    command: '/enroll-student',
    text: 'student-' + suffix + ' course-' + suffix,
    user_id: 'U_TEST_' + suffix,
    team_id: 'T_TEST',
    channel_id: 'C_TEST'
  };
  if (passcode) {
    payload.passcode = passcode;
  }

  var event = _mockDoPostEvent(_toFormBody(payload));
  var result = ctx.router.handle(event);
  Logger.log('[testSlashEnrollMock] ' + JSON.stringify(result));
}

/**
 * testWorkflowEnrollmentMock
 * Simulates `enrollment_requested` workflow webhook and runs full router path.
 */
function testWorkflowEnrollmentMock() {
  var ctx = _buildHostTestContext();
  var suffix = new Date().getTime();
  var payload = {
    workflow: 'lms_enrollment',
    eventType: 'enrollment_requested',
    requestId: 'wf_req_' + suffix,
    data: {
      studentId: 'wf-student-' + suffix,
      courseId: 'wf-course-' + suffix,
      actorId: 'WF_ACTOR'
    }
  };
  var event = _mockDoPostEvent(JSON.stringify(payload));
  var result = ctx.router.handle(event);
  Logger.log('[testWorkflowEnrollmentMock] ' + JSON.stringify(result));
```

```
}

/**
 * testAutomationMock
 * Simulates generic automation workflow webhook and runs full router path.
 */
function testAutomationMock() {
  var ctx = _buildHostTestContext();
  var suffix = new Date().getTime();
  var payload = {
    workflow: 'lms_automation',
    eventType: 'automation_ping',
    requestId: 'automation_req_' + suffix,
    data: {
      status: 'queued',
      actorId: 'AUTO_ACTOR'
    }
  };
  var event = _mockDoPostEvent(JSON.stringify(payload));
  var result = ctx.router.handle(event);
  Logger.log('[testAutomationMock] ' + JSON.stringify(result));
}

/**
 * testHostConfig
 * Verifies host config reads script properties without logging raw secrets.
 */
function testHostConfig() {
  var props = new ScriptPropertiesHelper();
  var hostConfig = new HostConfig(props);
  var keys = [
    'APP_PASSCODE',
    'SIGNING_SECRET',
    'WEBHOOK_SECRET',
    'API_KEY',
    'SLACK_BOT_TOKEN',
    'SLACK_SIGNING_SECRET',
    'SLACK_DEFAULT_CHANNEL'
  ];

  var values = {
    APP_PASSCODE: hostConfig.getAppPasscode(),
    SIGNING_SECRET: hostConfig.getSigningSecret(),
    WEBHOOK_SECRET: hostConfig.getWebhookSecret(),
    API_KEY: hostConfig.getApiKey(),
    SLACK_BOT_TOKEN: hostConfig.getSlackBotToken(),
    SLACK_SIGNING_SECRET: hostConfig.getSlackSigningSecret(),
    SLACK_DEFAULT_CHANNEL: hostConfig.getSlackDefaultChannel()
  };

  var redacted = {};
  keys.forEach(function(key) {
    var value = values[key] || '';
    redacted[key] = ScriptPropertiesHelper.shouldRedactKey(key)
      ? (value ? '[REDACTED_SET]' : '[REDACTED_EMPTY]')
      : value;
  });

  Logger.log('[testHostConfig] ' + JSON.stringify(redacted));
}

/**
 * testAuditWrite
 * Executes one mutation and verifies an audit row is written.
 */
function testAuditWrite() {
```

```
  var ctx = _buildHostTestContext();
  var auditTable = ctx.db.table('audit_logs');
  var before = auditTable.findAll().length;

  var suffix = new Date().getTime();
  var passcode = ctx.hostConfig.getAppPasscode();
  var payload = {
    command: '/enroll-student',
    text: 'audit-student-' + suffix + ' audit-course-' + suffix,
    user_id: 'U_AUDIT_' + suffix,
    team_id: 'T_AUDIT',
    channel_id: 'C_AUDIT'
  };
  if (passcode) {
    payload.passcode = passcode;
  }

  var event = _mockDoPostEvent(_toFormBody(payload));
  var result = ctx.router.handle(event);
  var after = auditTable.findAll().length;

  Logger.log('[testAuditWrite] ' + JSON.stringify({
    mutationOk: !!result.ok,
    auditRowsBefore: before,
    auditRowsAfter: after,
    auditRowDelta: after - before
  }));
}

'''
```